Handwritten Modi Character Recognition using Handcrafted Features

Ananya Viraj Choudhari
Department of Computer Engineering
Vishwakarma Institute of Technology
Pune, India
viraj.ananya24@vit.edu

Arya Bharat Patil
Department of Computer Engineering
Vishwakarma Institute of Technology
Pune, India
arya.patil24@vit.edu

***Abstract*—Modi is a cursive script that was used to write Marathi for several centuries, and a large share of the land, court and family records kept in Maharashtra's archives are written in it. Most recent work on reading handwritten Modi characters trains convolutional networks on about 90 images per class and reports accuracies that cannot be compared with each other. We went back to handcrafted features. Each character is described with HOG, Zernike moments and local binary patterns, the 2,261 values are reduced with LDA, and Random Forest, an RBF SVM, XGBoost and a stacked combination of the three are compared. On the public MODI-HChar dataset (57 classes) with a fixed test set of 11,400 images, the SVM reaches 86.4% with 500 training images per class and 88.9% with 2,000. Stacking did not beat the single SVM. Leaving one descriptor out at a time shows that HOG matters most and that Zernike moments add only 0.3 points once HOG and LBP are present. Grouped Kernel SHAP puts most of the evidence near the centre of the character, and the pairs the model confuses most, such as i and ii, differ by a stroke or two near the top. A plain CNN trained on exactly the same images is worse at 72 and 90 images per class and overtakes the handcrafted pipeline at 250.**

***Index Terms*—Modi script, handwritten character recognition, HOG, Zernike moments, local binary patterns, SHAP, explainable machine learning.**

I. Introduction

Modi is a cursive form of Nagari that was used for everyday Marathi writing, especially for revenue records, court papers and letters, until Devanagari replaced it in the twentieth century [1]. A great deal of this material survives in archives, but very few people can still read the script, so most of it has never been transcribed. Automatic character recognition would not solve that on its own, but it is a necessary first step.

Modi is not an easy script for a computer. Writers kept the pen on the paper, so letters hang from a shared top line and often run into each other. Several letters differ only by a small hook or loop, and labelled data is scarce. The public benchmark used by many recent papers has 46 classes with 90 images each [2], [3].

Most of the recent work trains convolutional networks on this small amount of data. Several of these papers report a large gap between training and validation accuracy [2], [4], and the accuracies are measured on different datasets, class counts and splits, so they cannot be ranked. Where classical models were tried as baselines, they were given raw pixels instead of features [5], which says little about what feature engineering can do. None of the papers we read explains individual predictions or lists which characters get mixed up.

This paper reports what a careful handcrafted pipeline can do on a public dataset with a fixed test set. Our contributions are:

1) a pipeline that fuses HOG, Zernike moments and LBP, reduces them with LDA, and compares Random Forest, SVM, XGBoost and stacking on 57 classes;
2) a leave-one-out ablation that measures what each descriptor adds on top of the other two;
3) grouped SHAP explanations for every prediction, together with a list of the most confused pairs and the regions that separate them;
4) a comparison with a plain CNN trained on identical splits, from 72 to 2,000 training images per class.

II. Related Work

Bhong and Kulkarni [2] compared a small CNN with a fine-tuned VGG16 on the 46-class benchmark and reported 89.5% validation accuracy for VGG16, with training accuracy about 8 points higher. Kolte and Gupta [3] added spatial and channel attention to a CNN to separate similar characters and reported 94.6%. Chandankhede et al. [4] attempted all 360 letter and vowel-sign combinations with ResNet152 and roughly 21 images per class; they report 97.38% average test accuracy but only 45.08% on validation, which points to heavy overfitting. Sonavane et al. [6] compared ResNet101, InceptionV3 and Xception with transfer learning.

Closer to our setting, Ekbote et al. [7] extracted VGG16 features and classified them with Random Forest and XGBoost, reaching 92% on characters and 93.3% on numerals, but the learned features cannot be interpreted and the classifiers were used one at a time. Mandke et al. [5] ran KNN, SVM and several CNNs on the same images; KNN and SVM scored 75% and 77% on raw pixels against 96% for the CNN. Among earlier classical studies, Anam and Gupta [8] combined Otsu binarisation with a Kohonen network on 22 characters (72.6%), and Kulkarni et al. [9] found that Zernike moments beat Hu moments on Modi digits (86.66% against 70%) on a set of 100 images.

Other groups work at the line or document level. Joseph and George [10] fine-tuned a CRNN on Modi text lines and report 88.14% character-level and 61% word-level accuracy, and Kausadikar et al. [11] released a document-level dataset with a vision-language model for direct transliteration to Devanagari. The review by Varpe and Sakhare [12] collects results from 2010 to 2022 and concludes that CNNs are the

strongest, but the highest numbers it lists come from different and mostly private datasets. What is missing is a fair test of handcrafted features on a public dataset, and some account of why a model decides what it does.

III. Dataset and Experimental Setup

We use MODI-HChar, the isolated-character part of the MODI-HDC collection on Kaggle [13]. After dropping unreadable files it has 575,912 images in 57 classes: 35 consonants, 12 vowels and 10 digits, with about 10,100 images per class. We counted these from disk rather than taking them from the dataset description.

The images of every class are shuffled once with a fixed seed. The first 200 images of each class form the test set (11,400 images), and nothing in the pipeline looks at them during training or model selection. Training sets take the next n images of each class, so a smaller training set is always contained in a larger one (Fig. 1). The main experiments use n = 500 (28,500 images); the learning curve uses n = 72, 90, 250, 500, 1,000 and 2,000. The value 72 matches the training size of the common 46 x 90 benchmark after a usual 80/20 split, but our data, classes and test protocol differ, so the comparison with published numbers is only approximate. Everything runs on a laptop CPU with a fixed random seed.

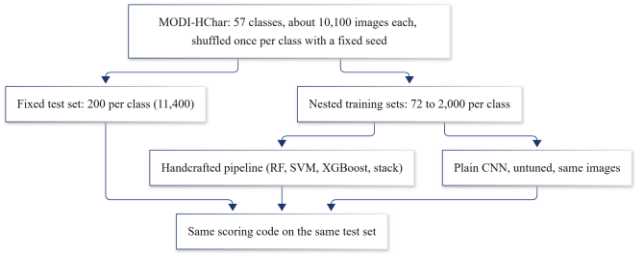


Fig. 1. Evaluation protocol. Both the handcrafted pipeline and the CNN are trained on the same nested training sets and scored by the same code on one fixed test set.

IV. Method

Fig. 2 shows the full pipeline. The implementation uses scikit-learn [14], scikit-image, mahotas, XGBoost and SHAP.

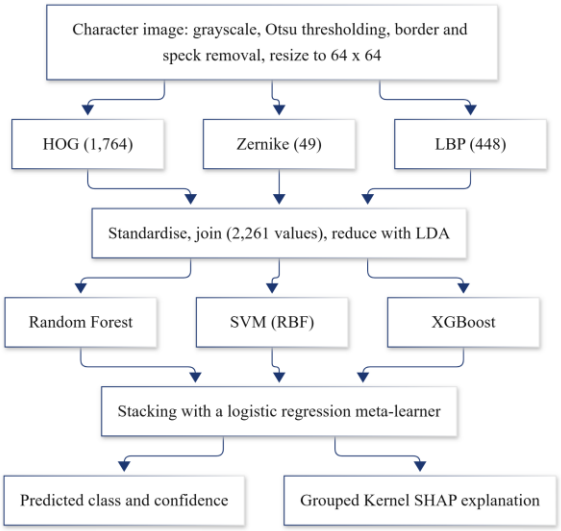


Fig. 2. The handcrafted recognition pipeline. The numbers in brackets are the lengths of the feature vectors.

A. Preprocessing

Each image is converted to grayscale, median filtered and binarised with Otsu's method [15]. Because the scans are not consistent about polarity, we treat the minority class of pixels as ink. Frame lines along the image border, long thin vertical grid lines and small specks are removed, the character is cropped to its ink, and it is scaled with its aspect ratio kept into a 64 x 64 image. We also tried deskewing, but it sheared some characters, so it is switched off.

B. Handcrafted Features

HOG [16] describes stroke direction. With 8 x 8 cells, 2 x 2 blocks and 9 orientation bins, the 64 x 64 image gives 7 x 7 blocks and 1,764 values. *Zernike moments* [17] describe the overall shape and do not change when the character rotates; we keep the magnitudes up to order 12, which gives 49 values. *LBP* [18] captures small local patterns such as stroke ends and junctions. We use uniform patterns at two scales (8 points at radius 1 and 16 points at radius 2) and pool their histograms over a 4 x 4 grid, which gives 448 values and keeps a rough idea of where each pattern occurs.

C. Fusion, Reduction and Classifiers

Every feature is standardised on the training set. Because HOG has 36 times more values than Zernike, each family f with d_f values is multiplied by

$$w_f = 1 / \sqrt{d_f} \qquad (1)$$

so that no family wins only through its length. The 2,261 values are then reduced with shrinkage LDA, which gives at most 56 dimensions for 57 classes. We chose LDA over PCA before looking at the test set: in 3-fold cross-validation on the training data, LDA reached 84.5% against 79.7% for PCA, averaged over Random Forest and SVM.

Four classifiers are trained on the reduced features. Random Forest [19] uses 400 trees. The SVM [20] uses an RBF kernel with C = 10. XGBoost [21] uses 200 rounds of depth-3 trees with a learning rate of 0.25; these settings are smaller than the defaults we started with, purely to keep CPU time reasonable. The stacking ensemble [22] trains a logistic

regression on the out-of-fold outputs of the other three (3 internal folds), using class probabilities from Random Forest and XGBoost and decision scores from the SVM.

D. Explanations

Explaining 2,261 separate numbers would not help anyone, so we group them into 35 groups that a person can point to: HOG pooled into 16 regions on a 4 x 4 grid, LBP into the same 16 regions, and Zernike moments into three bands (orders 0 to 4, 5 to 8, and 9 to 12). Kernel SHAP [23] then estimates how much each group moved the probability of the predicted class, with the mean training vector as the reference and 400 sampled coalitions per explanation. The contributions add up to the model output:

$$f(x) = \varphi_0 + \Sigma_g \varphi_g \quad (2)$$

where φ_0 is the output at the reference and φ_g is the contribution of group g. For the most confused pairs we also average the difference between the SHAP values of the true class and the look-alike class over their test images, which shows the regions that tell the two apart.

E. CNN Baseline

To see where handcrafted features stop being enough, we trained a plain CNN on the same 64 x 64 images: three blocks of 3 x 3 convolution (32, 64 and 128 filters), batch normalisation, ReLU and max pooling, then dropout of 0.5, a dense layer of 256 units and a 57-way softmax. It uses Adam with a learning rate of 0.001 and batches of 128. Ten percent of each training set is held out only to pick the number of epochs (early stopping with a patience of 6), after which the network is retrained on the whole training set for that many epochs, so it sees exactly the images the classical models see. We did not tune the architecture or add augmentation, since the classical side gets neither.

V. Results

A. Comparing the Classifiers

Table I shows the four models at 500 training images per class. The SVM is the best at 86.4%, and stacking comes second at 84.8%. We expected stacking to help, but it did not, and it stays below the SVM at every training size we tried. A likely reason is that its meta-learner is trained on base models that each saw only two thirds of the data, and two of those base models are clearly weaker than the SVM. Digits are the easiest group for every model and consonants the hardest. Of the 1,738 test errors made by the stack, 1,439 (83%) confuse one character with another of the same type.

TABLE I
Test accuracy at 500 training images per class (11,400 test images)

Model	Acc. (%)	Macro-F1	Digits	Vowels	Cons.
Random Forest	84.0	0.839	89.8	86.4	81.6
SVM (RBF)	86.4	0.863	92.7	88.9	83.7
XGBoost	83.8	0.838	90.0	86.1	81.2
Stacking	84.8	0.847	91.0	87.2	82.1

B. What Each Descriptor Adds

Fig. 3 shows the SVM with each descriptor alone, with each pair, and with all three. HOG alone already reaches 83.6% and LBP alone 79.3%, while Zernike moments alone manage only 54.0%. Removing one descriptor from the full set is more telling. Without HOG, accuracy falls by 4.9 points to 81.4%; without LBP it falls by 2.4 points; without Zernike it falls by only 0.3 points. So Zernike moments, which earlier work recommended for Modi digits [9], carry very little information that HOG and LBP do not already have. The weighting in (1) made no measurable difference after LDA (86.4% with and without it), which makes sense because LDA rescales the features on its own.

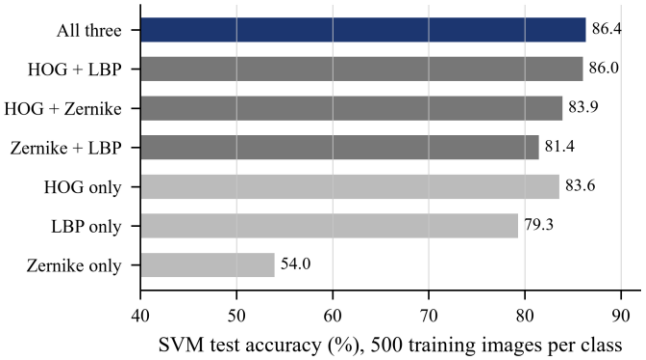


Fig. 3. SVM accuracy with single descriptors (light), pairs that leave one descriptor out (grey) and all three (dark).

C. Learning Curve Against the CNN

Fig. 4 plots test accuracy against training size. With 72 images per class the best handcrafted model reaches 80.4% while the CNN reaches 72.3%. At 90 images the gap narrows to 81.2% against 77.8%, and at 250 images the CNN is slightly ahead, 85.1% against 84.5%. The crossover therefore lies between 90 and 250 images per class. The handcrafted models keep improving with more data, up to 88.9% for the SVM at 2,000 images. The CNN runs at 500 images per class and above had not finished at the time of writing, so we do not claim how large its lead becomes. The cost is also very different on a CPU: each CNN point up to 250 images per class took about 64 to 96 minutes to train, while training all four handcrafted models at the same sizes took about 3 to 6.5 minutes.

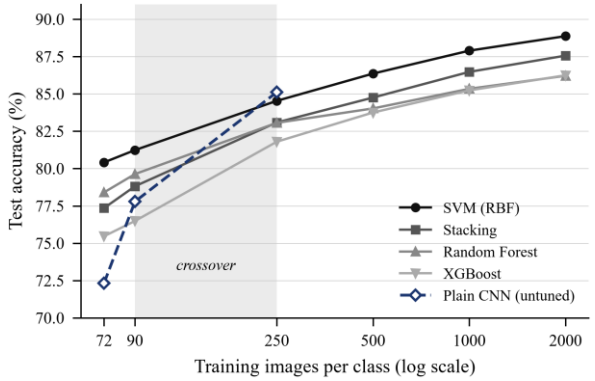


Fig. 4. Test accuracy against training images per class on a log scale. The shaded band marks the interval in which the CNN overtakes the best handcrafted model.

D. What the Explanations Show

We explained 171 test images (three per class). Fig. 5(a) compares each family's share of the inputs with its share of the total SHAP magnitude. HOG makes up 78% of the inputs and receives 66% of the evidence, LBP makes up 20% and receives 31%, and Zernike makes up 2% and receives 3%. LBP is therefore more useful per value than its size suggests, which agrees with the ablation. Fig. 5(b) shows where the HOG evidence comes from: the four central cells hold about half of it, while the top row holds only 17%. Our guess is that the headline runs across the top of nearly every character and so says little about which character it is.

To check that the explanations reflect what the model actually uses, we replaced the three highest-ranked groups of each image with their reference values. The predicted probability fell by 0.48 on average, compared with 0.06 when three random groups were replaced. The largest additivity error in (2) was below 10^{-6}.

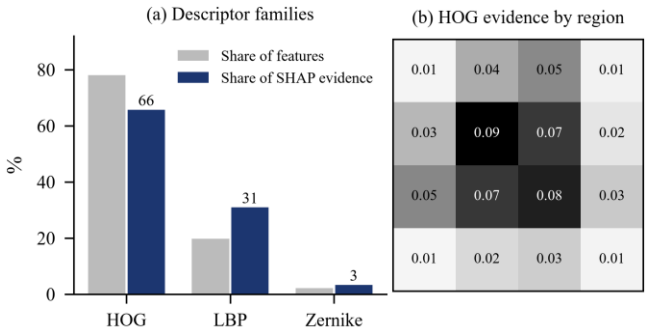


Fig. 5. (a) Share of input features and share of SHAP evidence for each descriptor family. (b) Mean absolute SHAP value of the 16 HOG regions on a 4 x 4 grid over the character.

Table II lists the five most confused pairs together with the region whose evidence separates them most. In every case it is a HOG region in the upper half of the character, which matches what a reader would look at: the extra hook of ii, for example, sits at the top right.

TABLE II
MOST CONFUSED PAIRS (STACKING ENSEMBLE, 400 TEST IMAGES PER PAIR)

Pair	Errors	Error rate	Main separating region
i / ii	91	22.8%	HOG, top centre-right
h / thha	89	22.2%	HOG, upper-middle centre-right
p / thha	54	13.5%	HOG, upper-middle centre-left
five / six	51	12.8%	HOG, lower-middle centre-right
k / m	46	11.5%	HOG, upper-middle centre-left

VI. DISCUSSION AND LIMITATIONS

For the amount of data most Modi papers work with, a handcrafted pipeline is not only competitive but better than a plain CNN, and it can say why it made each decision. The picture changes once a few hundred images per class are available. We should be careful with this conclusion, though. The CNN is deliberately untuned and gets no augmentation, so a tuned network would probably move the crossover to a smaller training size. The XGBoost settings were cut down for CPU time, which may explain part of its gap to the SVM.

The work also has clear limits. It recognises isolated characters only. Our app splits a line of separated characters, but on synthetic words whose characters touch along the headline the segmenter finds the right number of characters in only 42% of cases, so reading real cursive pages is still out of reach. The dataset comes from a limited group of writers, and the Devanagari names we show for each class have not yet been checked by a Modi reader.

VII. CONCLUSION

We built a Modi character recogniser from three handcrafted descriptors and classical classifiers, tested it on a public dataset with a fixed test set, and explained its predictions with grouped SHAP. The SVM reaches 86.4% at 500 images per class and 88.9% at 2,000. HOG and LBP do most of the work, stacking does not help, and the model mostly confuses characters that differ in a single stroke near the top. A plain CNN overtakes the pipeline between 90 and 250 images per class. The next steps are finishing the CNN runs at larger sizes, handling touching characters, and having the class labels verified by a Modi reader.

ACKNOWLEDGMENT

We thank our guide, Prof. Nilam Honmane, Department of Computer Engineering, Vishwakarma Institute of Technology, Pune, for her guidance throughout this project.

REFERENCES

[1] A. Pandey, "Proposal to encode the Modi script in ISO/IEC 10646," Unicode Technical Committee, Doc. L2/11-212R2, 2011.

[2] M. M. Bhong and N. Kulkarni, "Handwritten Modi script character recognition using CNN and VGG16 algorithms," Int. J. Adv. Res. Sci. Commun. Technol., 2025, doi: 10.48175/IJARSCT-25317.

[3] S. Kolte and B. Gupta, "Modi script character recognition using convolutional neural networks and attention-based CNN architecture," Int. J. Adv. Res. Sci. Commun. Technol., 2025, doi: 10.48175/IJARSCT-25309.

[4] C. Chandankhede, R. Sachdeo, U. Shrawankar, N. Wankhade, and G. Talmale, "Modi script handwritten character classification using drop-out induced incremental learning approach," J. Inf. Syst. Eng. Manage., vol. 10, no. 10s, 2025, doi: 10.52783/jisem.v10i10s.1403.

[5] S. Mandke, V. Pimprale, and P. Date, "Offline MODI script character recognition using deep learning techniques," Library Progress International, 2024.

[6] R. Sonavane et al., "Exploring ResNet101, InceptionV3, and Xception for Modi script character classification," Int. J. Intell. Syst. Appl. Eng., 2024.

[7] M. Ekbote, A. Jadhav, and D. D. Ambawade, "Implementing a hybrid deep learning approach to achieve classic handwritten alphanumeric MODI recognition," Int. J. Eng. Adv. Technol., 2022, doi: 10.35940/ijeat.A3846.1012122.

[8] S. Anam and S. Gupta, "An approach for recognizing Modi Lipi using Ostu's binarization algorithm and Kohenen neural network," Int. J. Comput. Appl., 2015, doi: 10.5120/19511-1128.

[9] S. A. Kulkarni, P. L. Borde, R. R. Manza, and P. L. Yannawar, "Recognition of handwritten MODI numerals using Hu and Zernike features," arXiv:1404.1151, 2014.

[10] S. Joseph and J. P. George, "Machine transliteration of handwritten MODI script to Devanagari using deep neural networks," Int. J. Comput., vol. 23, no. 4, 2024, doi: 10.47839/ijc.23.4.3540.

[11] H. Kausadikar, T. Kale, O. Susladkar, and S. Mittal, "Historic scripts to modern vision: A novel dataset and a VLM framework for transliteration of Modi script to Devanagari," arXiv:2503.13060, 2025.

[12] K. Varpe and S. R. Sakhare, "Review of character recognition techniques for MODI script," Indian J. Sci. Technol., vol. 16, no. 26, 2023, doi: 10.17485/IJST/v16i26.485.

[13] "MODI-HDC: Historical handwritten Modi script dataset" (MODI-HChar character subset), Kaggle. [Online]. Available: https://www.kaggle.com/

[14] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825-2830, 2011.

[15] N. Otsu, "A threshold selection method from gray-level histograms," IEEE Trans. Syst., Man, Cybern., vol. SMC-9, no. 1, pp. 62-66, 1979.

[16] N. Dalal and B. Triggs, "Histograms of oriented gradients for human detection," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit., 2005, pp. 886-893.

[17] A. Khotanzad and Y. H. Hong, "Invariant image recognition by Zernike moments," IEEE Trans. Pattern Anal. Mach. Intell., vol. 12, no. 5, pp. 489-497, 1990.

[18] T. Ojala, M. Pietikainen, and T. Maenpaa, "Multiresolution gray-scale and rotation invariant texture classification with local binary patterns," IEEE Trans. Pattern Anal. Mach. Intell., vol. 24, no. 7, pp. 971-987, 2002.

[19] L. Breiman, "Random forests," Mach. Learn., vol. 45, no. 1, pp. 5-32, 2001.

[20] C. Cortes and V. Vapnik, "Support-vector networks," Mach. Learn., vol. 20, no. 3, pp. 273-297, 1995.

[21] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Min., 2016, pp. 785-794.

[22] D. H. Wolpert, "Stacked generalization," Neural Netw., vol. 5, no. 2, pp. 241-259, 1992.

[23] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), 2017, pp. 4765-4774.